Table des matières

Exercices corrigés 00 — Mettre en place l'environnement 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 3
Mises en situation 3
Corrigé 5

Exercices corrigés 00 — Mettre en place l'environnement

Comment travailler ces exercices. Ce module est un module d'installation, et ses exercices le reflètent : ils portent moins sur le SQL que sur la compréhension de ce que vous venez de monter. Ils sont volontairement moins nombreux que ceux des autres cours. Faites-les une fois l'installation terminée, avant d'aborder le Cours 01 : s'ils vous coûtent, c'est le signe qu'une pièce de l'environnement vous échappe encore, et il vaut mieux le découvrir maintenant. Comptez deux heures et quart pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué `[OLTP]` porte sur PostgreSQL et la base `Adventureworks`. Un énoncé marqué `[OLAP]` porte sur DuckDB et l'entrepôt `adventureworksdw.duckdb`. Sans marque, la question vaut pour les deux.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer.

Questions courtes

1. Les deux mondes. Que veulent dire OLTP et OLAP, et quelle est la vocation de chacun en une phrase ?
2. Une entreprise, deux bases. AdventureWorks existe en version DW et en version OLTP. S'agit-il de deux entreprises, de deux périodes, ou de deux arrangements ? Expliquez.
3. Le rangement. Comment sont rangées les données dans chacune des deux versions, et quel problème chaque rangement cherche-t-il à résoudre ?
4. Le nombre de tables. La version OLTP compte 68 tables, la version DW beaucoup moins. Ce n'est pas parce qu'elle contient moins d'information : expliquez d'où vient l'écart.

© 2026 iAlexMG. Tous droits réservés.

5. [OLAP] Ce qu'il faut nommer. Pour atteindre une table de l'entrepôt, vous désignez trois niveaux. Nommez-les, du plus englobant au plus précis, et dites où chacun se trouve sur votre disque.
6. [OLAP] Une base dans un fichier. DuckDB ne demande ni service, ni port, ni mot de passe. Donnez deux conséquences pratiques de ce choix, une agréable et une contraignante.
7. Créer puis remplir. Le script de la Partie 2 fait deux choses de suite, dans cet ordre. Nommez-les et dites pourquoi elles sont distinctes.
8. Le format des données. Sous quelle forme AdventureWorks est-elle distribuée, et qu'est-ce que cela implique sur ce que fait réellement une installation de base de données d'exemple ?
9. La vérification. Après un chargement, quelle est la première requête à écrire, et pourquoi est-elle plus utile qu'un coup d'œil à la liste des tables ?
10. [OLTP] Le port. Quel port PostgreSQL utilise-t-il par défaut, et à quoi sert de le connaître ?
11. [OLTP] Le mot de passe. Pour quel utilisateur l'installeur vous demande-t-il un mot de passe, et pourquoi insiste-t-on pour que vous le notiez ?
12. [OLTP] La commande introuvable. Vous tapez `psql --version` et le terminal répond que la commande n'est pas reconnue. Est-ce que l'installation a raté ? Expliquez, dites ce qu'il faut faire, et pourquoi passer à pgAdmin ne suffit pas ici.
13. [OLTP] Le portage. AdventureWorks est née pour SQL Server. Pourquoi faut-il un portage pour la charger dans PostgreSQL, plutôt que de lancer le script Microsoft tel quel ?
14. La casse, des deux côtés. DuckDB ignore la casse des identifiants, PostgreSQL replie les identifiants non protégés en minuscules. Dites ce que chacun fait de `SELECT ListPrice FROM DimProduct`, et quelle erreur concrète vous attend au Cours 02 si vous l'oubliez.
1. Les deux clients. On propose pgAdmin, DBeaver ou `psql` pour PostgreSQL, et la ligne de commande pour DuckDB. Qu'est-ce qu'un client, et le choix change-t-il quoi que ce soit aux requêtes que vous écrivez ?
2. L'ordre de montage. Pourquoi la formation vous fait-elle monter l'entrepôt d'abord, et à partir de quel cours PostgreSQL devient-il indispensable ?
3. Les chemins relatifs. Le script de montage se lance depuis le dossier des fichiers `.csv`, et non depuis le dossier où il est rangé. Expliquez pourquoi, et dites quel message vous recevez si vous vous trompez de dossier.

Questions pratiques

Ces énoncés se tapent vraiment, dans votre environnement fraîchement monté.

P1. [OLAP] L'inventaire. Écrivez les deux commandes qui listent les tables chargées dans le schéma `dbo` de l'entrepôt, et vérifiez que celles du Cours 01 y sont.

P2. [OLAP] Le compte de contrôle. Écrivez, en une seule requête, le nombre de lignes de `DimProduct` et celui de `FactInternetSales`. Notez les deux nombres quelque part : ils vous serviront de repère tout au long de la formation.

© 2026 iAlexMG. Tous droits réservés.

P3. [OLTP] Le même inventaire, de l'autre côté. Listez les schémas de la base `Adventureworks`, puis les tables du schéma `production`. Comparez le nombre de tables avec celui de l'entrepôt.

P4. [OLTP] Le compte de contrôle, de l'autre côté. Comptez les lignes de `production.product` et de `sales.salesorderdetail`.

P5. La même question, deux fois. Comptez les produits sur les deux bases, avec la requête appropriée à chacune. Les deux nombres sont proches sans être identiques : proposez au moins deux explications possibles avant de lire le corrigé.

P6. (avancé) L'ordre de grandeur. Sur chacune des deux bases, trouvez la table qui contient le plus de lignes. Puis dites ce que la réponse vous apprend sur la vocation de chaque base.

Diagnostics

On vous décrit une situation d'installation qui tourne mal. Dites ce qui se passe et comment vous vous y prenez.

E1. [OLAP] Le script de montage s'est déroulé sans une erreur, et le tableau final affichait bien 60 398 ventes. Vous rouvrez l'entrepôt le lendemain et tapez `SELECT COUNT(*) FROM DimProduct;` : le moteur répond `Table with name DimProduct does not exist!`. Donnez la cause la plus probable, la commande qui règle le problème, et une deuxième cause possible si la première ne s'applique pas.

E2. [OLAP] Une collègue vous écrit : son script Python refuse d'ouvrir l'entrepôt sur un `IO Error: Cannot open file`, alors que le fichier est bien là et qu'elle a les droits dessus. Elle en conclut que le fichier est corrompu. Dites ce qui se passe le plus probablement, pourquoi son explication est peu plausible, et ce que le message lui disait déjà.

E3. [OLTP] Vous avez lancé `install.sql` et le chargement s'est arrêté au milieu avec une erreur de fichier introuvable. Vous relancez le script : cette fois, ce sont des erreurs de table déjà existante qui défilent. Expliquez ce qui se passe, dites pourquoi relancer bêtement ne peut pas marcher, et donnez la marche à suivre. Puis dites pourquoi le script de la Partie 2, lui, se rejoue sans dommage.

Questions à développer

Un court paragraphe de raisonnement, pas un mot juste.

A. Pourquoi une base d'exemple. On aurait pu enseigner le SQL sur trois tables inventées de dix lignes chacune. Expliquez ce qu'apporte une vraie base d'exemple comme AdventureWorks, et ce qu'elle coûte. Traitez au moins l'installation, le réalisme des données et la transférabilité de ce que vous apprenez vers votre futur emploi.

B. Le local et le distant. DuckDB tient dans un fichier sur votre disque ; les entrepôts d'entreprise, Snowflake, BigQuery, Databricks, tournent sur des machines que vous ne verrez jamais. Décrivez ce que cette différence change concrètement : sur ce que vous contrôlez, sur ce qui peut casser, sur ce que ça coûte, et sur ce qui se passe si vous n'y touchez pas pendant deux mois. Dites ensuite ce qu'elle ne change pas, et pourquoi c'est ce qui justifie d'apprendre ici.

Mises en situation

© 2026 iAlexMG. Tous droits réservés.

On décrit une situation et on vous demande une décision, pas une commande.

M1. Le poste de travail partagé. Vous devez faire suivre cette formation à trois personnes de votre équipe, sur des postes gérés par le service informatique, où personne n'a les droits d'administrateur et où l'installation de logiciels demande une demande formelle de deux semaines.

Décrivez la solution que vous proposeriez, en traitant séparément le cas de DuckDB et celui de PostgreSQL. Dites ce que vous perdez par rapport à l'installation décrite dans ce module, et à quel cours cette perte se fera sentir pour la première fois. Terminez en disant ce que vous demanderiez au service informatique, en une seule demande, pour que le reste tienne.

M2. Le jour où il faut porter tout ça. Six mois après la formation, votre employeur met en service un entrepôt Snowflake et vous demande d'y porter les requêtes d'analyse que vous aviez écrites sur votre poste. Vous avez une centaine de requêtes, un entrepôt local de 25 Mo, et deux semaines.

Décrivez ce que vous faites, dans l'ordre. Distinguez trois catégories : ce qui se transporte tel quel, ce qui demande une retouche connue d'avance, et ce qui n'a pas d'équivalent et doit être repensé. Dites ensuite quelle habitude prise dès ce Cours 00 rend ce portage possible en deux semaines plutôt qu'en deux mois, et pourquoi c'est une leçon qui dépasse largement le cadre d'une formation.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés.

Questions courtes

1. OLTP veut dire *Online Transaction Processing*, le traitement transactionnel : faire tourner l'entreprise au quotidien, avec beaucoup de petites lectures et écritures en continu. OLAP veut dire *Online Analytical Processing*, le traitement analytique : comprendre l'entreprise, avec peu de requêtes mais qui balaient de gros volumes.

2. Deux arrangements de la même entreprise, sur la même période. Rien n'est inventé de part et d'autre : ce sont les mêmes ventes des mêmes vélos, réorganisées pour deux usages incompatibles.

3. La version OLTP est normalisée : chaque information est stockée une seule fois, dans une table qui lui est propre, et les tables sont reliées par des clés. Elle cherche à éviter la redondance et les incohérences quand des milliers d'écritures arrivent en continu. La version DW est dénormalisée en étoile : le contexte d'une vente est rassemblé autour d'elle. Elle cherche à éviter les jointures quand une requête balaie des millions de lignes.

4. L'écart vient de la normalisation, pas du contenu. Le monde transactionnel éclate chaque notion en autant de tables qu'il y a de relations à représenter, avec des tables de liaison, des tables de référence, des tables d'historique. L'entrepôt replie tout cela dans quelques dimensions larges : `DimProduct` réunit le produit, son modèle et leurs tables de traduction, que le transactionnel sépare. Le repliement n'est pas total pour autant : la sous-catégorie et la catégorie restent deux tables à part, que le Cours 01 joindra, et c'est ce qui fait de cette branche un flocon plutôt qu'une étoile stricte.

5. `[OLAP]` La base, le schéma, la table : `adventureworksdw`, puis `dbo`, puis `DimProduct`. La base est un fichier de votre disque, `Projet_SQL/adventureworksdw.duckdb`, et son nom vient du nom du fichier. Le schéma et la table, eux, n'existent que dedans : ce ne sont pas des dossiers, c'est du rangement interne au fichier.

6. `[OLAP]` L'agréable : l'entrepôt se copie, se sauvegarde, s'envoie et s'efface comme un document, rien ne tourne en tâche de fond, et il n'y a ni compte à ouvrir, ni échéance, ni facture. Le contraignant : un fichier ouvert en écriture n'accepte qu'un seul processus à la fois, donc deux programmes ne peuvent pas travailler dessus en même temps. C'est exactement la contrepartie de l'absence de serveur, dont le métier est précisément d'arbitrer entre plusieurs clients.

7. Créer les tables, qui sont des structures vides, puis y charger les données. Les deux gestes sont distincts parce qu'ils relèvent de deux familles de commandes différentes, la définition puis la manipulation. Vous l'apprendrez au Cours 01 sous le nom de DDL et de DML, et vous vous rendrez compte que vous l'aviez déjà fait ici sans le savoir. La raison concrète, dans notre cas, est qu'il faut avoir nommé les colonnes avant de verser des fichiers qui ne les nomment pas.

8. Sous forme de fichiers plats, un par table, remplis de lignes fictives. Ce que cela implique est utile à retenir : installer une base d'exemple, ce n'est pas installer un logiciel, c'est créer des structures vides puis y verser des fichiers. La même mécanique servira le jour où vous chargerez de vraies données.

© 2026 iAlexMG. Tous droits réservés.

9. Un compte de lignes, `SELECT COUNT(*)`, sur une ou deux tables représentatives. Elle est plus utile qu'un coup d'œil à la liste des tables pour deux raisons. La liste montre l'existence de la table, pas son contenu : une table créée mais vide y ressemble exactement à une table pleine. Et une requête vérifie du même coup que la base ouverte et le schéma courant sont les bons, ce qu'aucune liste d'objets ne vous dira.

10. `[OLTP]` Le port 5432. Le connaître sert dès qu'on sort du chemin par défaut : configurer un client, ouvrir un pare-feu, faire tourner deux versions de PostgreSQL en parallèle, ou se connecter à un conteneur Docker qui expose autre chose.

11. `[OLTP]` Pour l'utilisateur `postgres`, le superutilisateur de l'instance. On insiste parce qu'il n'y a pas de mécanisme de récupération simple : ce mot de passe ne se retrouve pas, il se réinitialise, en modifiant un fichier de configuration et en redémarrant le service. Une demi-heure perdue pour n'avoir pas noté une ligne.

12. `[OLTP]` Non, l'installation n'a pas raté. L'installeur EDB n'ajoute pas le dossier `bin` au chemin de recherche de Windows, donc le terminal ne trouve pas l'exécutable, ce qui n'a rien à voir avec le serveur, qui tourne comme un service. Il faut donc ajouter ce dossier au chemin de recherche, puis rouvrir un terminal (étape 7 du guide d'installation du parcours). pgAdmin ne remplace pas ce réglage : il sert à écrire des requêtes, alors que le chargement d'AdventureWorks passe par `createdb` et `psql`, deux programmes qu'on lance au terminal.

13. `[OLTP]` Parce que le script de Microsoft est écrit en T-SQL, le dialecte de SQL Server, avec des types et des commandes que PostgreSQL ne connaît pas. Le portage traduit la structure et prépare les fichiers de données. C'est une bonne illustration de la nuance du Cours 01 : le SQL de base est le même partout, mais un script d'administration complet est presque toujours propre à un moteur.

14. Les deux moteurs acceptent la requête, pour deux raisons différentes. DuckDB compare les identifiants sans tenir compte de la casse, et garde pour l'affichage celle qui a été déclarée à la création : l'en-tête du résultat dit `ListPrice`. PostgreSQL, lui, replie tout identifiant non protégé en minuscules avant de chercher, et la colonne du portage s'appelle bel et bien `listprice` : l'en-tête du résultat dit `listprice`.

L'erreur qui vous attend au Cours 02 n'est donc pas celle qu'on croit. Ce n'est pas d'écrire `ListPrice`, c'est de mettre des guillemets doubles autour : `"ListPrice"` demande à PostgreSQL la colonne exactement ainsi orthographiée, qui n'existe pas, et vous obtenez « la colonne "ListPrice" n'existe pas » sur une table qui contient manifestement un prix de liste. La règle se retient en une phrase : sans guillemets, chaque moteur applique sa règle de repliement ; avec guillemets, il faut l'orthographe exacte. Une colonne du portage oblige d'ailleurs à les mettre, `sales.salesterritory` et sa colonne `"group"`, parce que `GROUP` est un mot réservé du langage ; c'est le Cours 06 qui la croise.

15. Un client est le programme depuis lequel vous écrivez et envoyez vos requêtes ; il ne calcule rien lui-même, il parle au moteur. Le choix ne change rien aux requêtes : le même SQL envoyé depuis pgAdmin, DBeaver ou `psql` produit le même résultat. Il change le confort, l'affichage des résultats et les outils annexes, pas la sémantique. Nuance pour DuckDB : sa ligne de commande accepte en plus quelques commandes à elle, qui commencent par un point (`.quit`, `.mode`, `.timer`). Ce ne sont pas du SQL, et elles ne passeront pas ailleurs.

16. Parce que la formation est analytique d'abord et que tous les cours ont une voie entrepôt : sans elle, on ne peut rien faire. PostgreSQL devient indispensable au Cours 02, où commence le double chemin, chaque cours ayant dès lors un laboratoire sur chaque terrain.

17. Parce qu'un chemin de fichier écrit dans un script SQL est résolu par rapport au dossier courant du terminal, et non par rapport à l'endroit où le script est rangé. Le script dit `FROM 'DimProduct.csv'` sans préfixe : le moteur cherche ce fichier là où vous êtes. Si vous vous trompez de dossier, vous

© 2026 iAlexMG. Tous droits réservés.

obtenez `IO Error: No files found that match the pattern "FactInternetSales.csv"`, sur la première table chargée. La leçon est générale : un chemin relatif se lit toujours depuis le répertoire courant du processus.

Questions pratiques

P1. [OLAP] Deux commandes, parce que la première n'affiche rien tant que le schéma courant n'est pas le bon :

```
USE dbo;
SHOW TABLES;
```

`SHOW TABLES` ne montre que le schéma courant, qui vaut `main` à l'ouverture. Sans le `USE`, la liste revient vide alors que les huit tables sont bien là. Une variante qui ne dépend pas du contexte, et qui marche sur à peu près tous les moteurs :

```
SELECT table_name FROM information_schema.tables WHERE table_schema = 'dbo' ORDER BY table_name;
```

P2. [OLAP]

```
SELECT (SELECT COUNT(*) FROM dbo.DimProduct)         AS nb_produits,
       (SELECT COUNT(*) FROM dbo.FactInternetSales)  AS nb_ventes;
```

Vous devriez obtenir 606 produits et 60 398 ventes. Ces deux nombres sont vos repères : si une requête d'un cours ultérieur vous rend un compte très différent sans raison, c'est souvent qu'une jointure a éclaté les lignes.

P3. [OLTP]

```
SELECT schema_name FROM information_schema.schemata ORDER BY schema_name;

SELECT table_name FROM information_schema.tables
WHERE table_schema = 'production'
ORDER BY table_name;
```

Le schéma `production` compte à lui seul plus de tables que tout l'entrepôt. C'est la normalisation en action, et c'est visible avant même d'avoir écrit une requête analytique.

P4. [OLTP]

```
SELECT (SELECT COUNT(*) FROM production.product)        AS nb_produits,
       (SELECT COUNT(*) FROM sales.salesorderdetail)    AS nb_lignes_commande;
```

504 produits et 121 317 lignes de commande, les deux comptes de contrôle de l'étape 4.2 du labo.

P5. Les deux comptes sont 606 pour l'entrepôt et 504 pour le transactionnel.

Deux explications tiennent debout, et il fallait les envisager avant de connaître la réponse. Le périmètre peut différer : l'entrepôt ne charge pas forcément tout le catalogue de la source, ou en ajoute. Ou bien le grain diffère : une ligne de la dimension ne représente peut-être pas exactement un produit.

© 2026 iAlexMG. Tous droits réservés.

C’est la seconde qui est la bonne, et elle vaut la peine d’être vue dès maintenant. `DimProduct` contient 606 lignes pour 504 produits distincts, comme le montre cette requête :

```
SELECT COUNT(*) AS lignes, COUNT(DISTINCT ProductAlternateKey) AS produits_reels
FROM dbo.DimProduct;
```

`ProductAlternateKey` est le code du produit dans le système source, et il se répète. La dimension garde en effet plusieurs versions d’un même produit, chacune valable sur une période, avec des colonnes de date de début, de date de fin et un statut. C’est ce qu’on appelle une dimension à évolution lente : quand le prix d’un produit change, l’entrepôt n’écrase pas l’ancienne ligne, il en ouvre une nouvelle, ce qui permet de recalculer un rapport de l’an dernier avec les prix de l’an dernier.

La conséquence pratique est immédiate et vous suivra toute la formation : compter les lignes d’une dimension ne compte pas des objets du monde réel. Quand vous voudrez le nombre de produits, ce sera `COUNT(DISTINCT ProductAlternateKey)`, ou un filtre sur les lignes courantes.

P6. `(avancé)`

Sur l’entrepôt, c’est la table de faits `FactInternetSales`, avec ses 60 398 lignes, devant `DimCustomer` et ses 18 484. Sur le transactionnel, c’est `sales.salesorderdetail` avec ses 121 317 lignes. Le classement se lit d’un coup côté DuckDB :

```
SELECT table_name, estimated_size FROM duckdb_tables()
WHERE schema_name = 'dbo' ORDER BY estimated_size DESC;
```

Ce que la réponse apprend est vrai des deux côtés, et c’est rassurant : dans les deux mondes, la plus grosse table est celle des événements, pas celle des choses. Il y a toujours beaucoup plus de ventes que de produits. La différence est dans ce qui entoure cette table. Sur le transactionnel, les lignes de commande sont entourées de dizaines de petites tables normalisées ; sur l’entrepôt, la table de faits est entourée de quelques dimensions larges. Même centre de gravité, deux façons de l’organiser.

Diagnostics

E1. `[OLAP]` La cause la plus probable est le schéma courant. Une session DuckDB s’ouvre sur le schéma `main`, qui est vide : les tables sont dans `dbo`, et rien ne le rappelle au démarrage. Le moteur dit donc en toute rigueur que `DimProduct` n’existe pas, sous-entendu là où il a regardé. La commande qui règle le problème est `USE dbo;`, à taper au début de chaque session, ou l’écriture du nom complet, `dbo.DimProduct`.

La deuxième cause, si le `USE` ne change rien, est que vous n’avez pas ouvert le bon fichier. Une commande `duckdb` suivie d’un nom de fichier qui n’existe pas ne se plaint pas : elle crée une base neuve et vide, du nom demandé, dans le dossier courant. Une faute de frappe ou un mauvais dossier suffit donc à vous donner un entrepôt parfaitement vide et parfaitement silencieux. Le test : `SHOW TABLES` après le `USE`, puis un coup d’œil au dossier, pour voir si un second fichier `.duckdb` de quelques kilo-octets y traîne à côté du vrai, qui pèse 25 Mo.

E2. `[OLAP]` Un fichier DuckDB ouvert en écriture n’accepte qu’un seul processus, et c’est presque toujours l’explication : elle a laissé une ligne de commande DuckDB ouverte sur l’entrepôt, ou une session Python précédente qui ne s’est jamais fermée, ou son éditeur qui l’a ouvert pour l’inspecter. Le second programme est refusé, quels que soient ses droits.

© 2026 iAlexMG. Tous droits réservés.

Son explication est peu plausible pour une raison simple : un fichier corrompu se signale au moment de la lecture, par une erreur de format ou de somme de contrôle, pas par un refus d'ouverture. Et le message lui disait déjà l'essentiel, puisque DuckDB nomme le programme fautif et son numéro de processus : `File is already open in …`. Le réflexe à lui transmettre est de lire le message en entier avant de conclure, parce qu'un moteur qui connaît la cause la donne presque toujours.

E3. `[OLTP]` Le premier échec a laissé la base dans un état partiel : une partie des tables a été créée, une partie des données chargée, et le script s'est arrêté là. Relancer un script d'installation qui commence par des `CREATE TABLE` ne peut pas marcher, parce que ces commandes échouent sur les objets déjà présents. Et comme les erreurs défilent, on ne voit pas que la vraie faute, la première, est toujours là.

La marche à suivre est de repartir d'une base propre, et non de réparer. On supprime la base et on la recrée vide, puis on corrige d'abord la cause de l'erreur initiale, un fichier au mauvais endroit dans la quasi-totalité des cas, avant de relancer le script en entier.

La suppression ne se fait pas depuis la base elle-même : PostgreSQL refuse de supprimer celle à laquelle la session est connectée, et répond « cannot drop the currently open database ». On sort donc du client, et on passe par les deux programmes du labo, qui travaillent depuis le terminal :

```
dropdb -U postgres Adventureworks
createdb -U postgres Adventureworks
```

Depuis une session `psql` déjà ouverte, il faut d'abord changer de base, par `\c postgres`, avant de lancer `DROP DATABASE "Adventureworks";` puis `CREATE DATABASE "Adventureworks";`.

Le script de la Partie 2 se rejoue sans dommage pour une raison d'écriture, et non de moteur : ses commandes de création sont des `CREATE OR REPLACE TABLE`, qui refont la table à neuf qu'elle existe ou non, et le chargement suit dans la foulée. Il ne suppose donc aucun état de départ. C'est ce qu'on appelle un script idempotent, et c'est une propriété qu'on écrit exprès : la notion revient au Cours 03, avec le `MERGE`, et elle est au cœur de tout ce qui se recharge chaque nuit en production.

La leçon générale dépasse cet incident : devant un chargement interrompu par un script qui n'est pas idempotent, le réflexe correct est toujours de revenir à un état connu, jamais de relancer par-dessus.

Questions à développer

A. Une vraie base d'exemple apporte trois choses qu'aucune table inventée ne donne.

Elle apporte le désordre. Des couleurs manquantes, des produits sans sous-catégorie, des années incomplètes, des codes au lieu de libellés : ce sont exactement les situations qui font échouer les requêtes des débutants, et sur trois tables propres de dix lignes, elles n'apparaissent jamais. La moitié des pièges enseignés dans cette formation ne seraient tout simplement pas visibles.

Elle apporte l'échelle. Une jointure qui éclate le nombre de lignes, un `COUNT(*)` qui ne compte pas ce qu'on croit, une requête lente : rien de tout cela ne se manifeste sur dix lignes. On peut énoncer la règle, on ne peut pas la faire ressentir.

© 2026 iAlexMG. Tous droits réservés.

Elle apporte enfin la transférabilité, et c'est l'argument décisif dans une formation qui vise un emploi. AdventureWorks est la base d'exemple la plus connue du monde Microsoft : les exemples que vous croiserez dans la documentation, dans les tutoriels et parfois dans les questions d'entretien s'y réfèrent. Le vocabulaire que vous apprenez ici est celui que d'autres emploieront devant vous.

Ce qu'elle coûte est réel. Le téléchargement et le montage prennent du temps, et c'est du temps passé avant d'avoir écrit une requête. La base est plus grosse que nécessaire, avec des dizaines de tables qu'on n'ouvrira jamais, ce qui noie le débutant. Et son modèle porte des décisions de conception discutables, qu'on finit par prendre pour des normes. C'est un échange, et il est très favorable : le coût se paie une fois, au Cours 00, et le bénéfice court sur les douze modules.

B. La différence porte sur quatre choses, et il faut la retourner à la fin.

Sur ce que vous contrôlez. En local, vous avez tout : la version du moteur, le fichier, les copies, le moment où vous le rechargez. Sur une plateforme distante, vous n'avez que ce que l'interface expose, et la contrepartie est que vous ne pouvez pas non plus casser ce que vous ne voyez pas.

Sur ce qui peut casser. En local, ce qui casse est de votre côté : un disque plein, un fichier verrouillé par un autre programme, une mise à jour de Windows, un antivirus qui bloque un exécutable. À distance, ce qui casse est la connexion, l'authentification, ou le compte lui-même. Ce sont deux catalogues de pannes entièrement disjoints, et deux réflexes de diagnostic différents.

Sur le coût. En local, le coût est fixe et déjà payé : votre machine tourne de toute façon, et un entrepôt de 25 Mo ne se remarque pas. À distance, il est proportionnel à l'usage, ce qui est confortable tant qu'on ne laisse rien tourner, et douloureux le jour où l'on oublie une ressource active. La suspension automatique n'y est pas un détail de configuration, c'est le garde-fou principal.

Sur l'abandon de deux mois, enfin, et c'est le point le plus concret. Votre entrepôt local sera exactement là où vous l'avez laissé : c'est un fichier, il ne bouge pas. Un compte d'essai distant, lui, aura peut-être expiré, et tout ce qu'il contenait avec. Le nuage n'oublie pas vos données, mais il peut oublier votre droit d'y accéder, et la différence est la même en pratique.

Ce que la différence ne change pas, c'est le langage, et c'est ce qui justifie d'apprendre ici. Une requête analytique écrite sur DuckDB, avec ses jointures, son fenêtrage, ses CTE et ses agrégats, est la même sur les grandes plateformes, à quelques fonctions près que les encadrés « Portabilité » signalent au fil des cours. Ce qui diffère, ce sont les gestes d'administration, les chargements, la gestion des accès, la facturation : le métier de la plateforme, pas celui du langage.

Mises en situation

M1. Le poste de travail partagé.

Les deux moteurs ne posent pas du tout le même problème, et c'est précisément ce qui rend la situation gérable.

DuckDB est un exécutable unique, sans service, sans port et sans clé de registre. Sur un poste verrouillé, on le pose dans un dossier utilisateur et on l'appelle par son chemin, sans droits d'administrateur ; le paquet Python, installé dans un environnement personnel, fait le même travail depuis un notebook ou un script. Les fichiers d'entrepôt vivent dans le dossier de chacun. Il reste à vérifier que l'antivirus de l'entreprise laisse s'exécuter un binaire posé à la main, ce qui n'est pas acquis partout, et que le téléchargement des données de Microsoft n'est pas bloqué.

© 2026 iAlexMG. Tous droits réservés.

PostgreSQL demande une vraie installation, un service et un port, donc il bloque. Deux contournements existent. Le premier est une instance PostgreSQL hébergée, sur un service gratuit ou sur un serveur de l'équipe, à laquelle chacun se connecte depuis DBeaver en version portable. Le second est une version portable de PostgreSQL, qui s'exécute depuis un dossier utilisateur sans droits d'administrateur, moins confortable mais suffisante pour une formation.

Ce que vous perdez, si aucun des deux contournements n'aboutit, c'est la voie OLTP de tous les labos. Le manque se fera sentir dès le Cours 02, où le double chemin commence, et pas au Cours 10 comme on pourrait le croire : les cours 02 à 09 ont chacun un laboratoire transactionnel, et ce sont eux qui font la valeur du contraste. On peut suivre la formation sans, en lisant la voie OLTP au lieu de la taper, mais on perd la moitié de ce qu'elle enseigne.

La demande unique que j'adresserais au service informatique : l'accès à une instance PostgreSQL partagée pour l'équipe, avec un utilisateur par personne et une base par personne. C'est une seule demande, elle ne touche à aucun poste de travail, elle est plus facile à accorder qu'une installation locale, et elle sert bien au-delà de la formation.

M2. Le jour où il faut porter tout ça.

Dans l'ordre, et l'ordre compte, parce qu'il va du certain vers l'incertain.

D'abord, rassembler et faire l'inventaire. Les requêtes sont des fichiers `.sql` rangés par cours dans un dépôt Git : on les ouvre, on les classe, on note pour chacune la table qu'elle lit et les fonctions qu'elle emploie. Une demi-journée, et c'est la seule étape qui ne se délègue pas.

Ensuite, trier en trois tas. Le premier, le plus gros, se transporte tel quel : les jointures, les filtres, les agrégats, le `GROUP BY`, les CTE, le fenêtrage, `UNION` et ses parents, la plupart des sous-requêtes. C'est le SQL normalisé, et c'est précisément ce que cette formation enseigne. Le deuxième demande une retouche connue d'avance, parce que chaque moteur a ses fonctions à lui : les calculs de dates, la manipulation du JSON, le pivot dynamique, quelques fonctions de texte. La retouche est mécanique une fois qu'on sait laquelle ; les encadrés « Portabilité » des cours sont exactement cette liste. Le troisième tas n'a pas d'équivalent et se repense : les particularités de stockage et de chargement, les commandes d'administration, et tout ce qui touche au fichier local, qui n'a plus de sens sur une plateforme partagée.

Enfin, remonter les données et rejouer. Les fichiers de départ sont les mêmes, le script de montage se retraduit une fois, et le contrôle est déjà écrit : les comptes de la Partie 3. Une requête qui rend 60 398 ventes des deux côtés est portée ; une qui rend autre chose ne l'est pas.

L'habitude prise dès ce Cours 00 est celle qui rend tout le reste possible : vos requêtes sont du code, pas des brouillons. Un fichier par sujet, sur votre disque, versionné, et l'outil du moteur réduit à un endroit où l'on colle du code pour l'exécuter. Un travail qui vit ainsi se relit, se compare, se cherche et se porte. La leçon dépasse la formation parce qu'elle est le premier réflexe de l'ingénierie analytique moderne : ce qui vit uniquement dans l'interface d'un outil, requête, transformation ou tableau de bord, n'est ni relisible, ni partageable, ni récupérable, et disparaît avec l'outil.

© 2026 iAlexMG. Tous droits réservés.